

# CSCI 232

## 232 Algorithms & Data Structures

**LECTURE: 9**

J. L. Pach

# TODAY'S AGENDA

- Logarithmic and Exponential Rules
- Asymptote
- Asymptotic notation
  - Θ-notation
  - O-notation
  - Ω-notation

# LOGARITHMIC

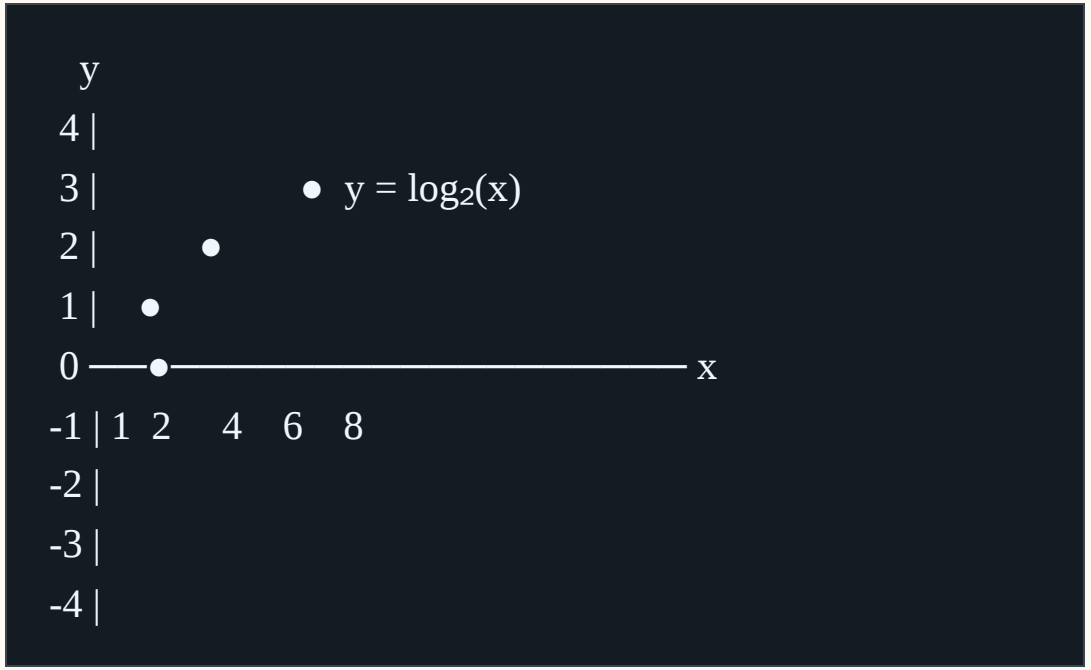
## and Exponential Rules

# BINARY LOGARITHM

In mathematics, the **binary logarithm** ($\log_2 x$) is the power to which the number 2 must be raised to obtain the value x. That is, for any real number y:

- $y = \log_2 x \equiv 2^y = x$
- $\log_2 x^c \equiv c \cdot \log_2 x$
- $\log x \equiv \log_2 x \quad (\text{CS})$
- $\lg x \equiv \log_{10} x \quad (\text{from math})$

$$f(x) = \log_2(x) = y$$
$$f(8) = \log_2(8) = 3$$



# The Fundamentals 1/3

1. **The Product Rule:** The logarithm of a product of two numbers, **a** and **c**, is equal to the **sum** of the logarithms of those numbers.

$$\log_b(ac) = \log_b a + \log_b c$$

2. **The Quotient Rule:** The logarithm of a quotient (a division) is equal to the **difference** between the logarithms of the numerator (a) and the denominator (c).

$$\log_b\left(\frac{a}{c}\right) = \log_b a - \log_b c$$

3. **The Power Rule:** When the number inside the logarithm (a) is raised to a power (c), that exponent can be moved and multiplied **in front** of the logarithm.

$$\log_b(a^c) = c \cdot \log_b a$$

# The Fundamentals 2/3

4. **The Change-of-Base Formula:** This formula allows you to convert a logarithm from an existing base (b) to a new, more convenient base (c). This is done by taking the logarithm of the original number (a) and dividing it by the logarithm of the old base (b), both using the new base c.

$$\log_b a = \frac{\log_c a}{\log_c b}$$

5. **Logarithmic exponential identity:** Logarithmic Exponential Identity: This identity shows that when raising a base b to a logarithm with a different base c, you can swap the original base b with the logarithm's argument a, and the result remains the same.

$$b^{\log_c a} = a^{\log_c b}$$

6. **The Power of a Power Rule:** When an exponential expression (ba) is raised to another power (c), you keep the base (b) and multiply the exponents (a and c).

$$(b^a)^c = b^{ac}$$

# The Fundamentals 3/3

7. **The Product Rule for Exponents:** To multiply two powers with the same base (b), you keep the base and add the exponents (a and c).

$$b^a b^c = b^{a+c}$$

8. **The Quotient Rule for Exponents:** To divide two powers with the same base (b), you keep the base and subtract the exponent of the denominator (c) from the exponent of the numerator (a).

$$\frac{b^a}{b^c} = b^{a-c}$$

# Examples 1/2

1. $\log(2n \log n) = 1 + \log n + \log(\log n)$ — 1. Rule 1 (twice)
2. $\log\left(\frac{n}{2}\right) = \log n - \log 2 = \log n - 1$ — 2. Rule 2
3. $\log \sqrt{n} = \frac{\log n}{2}$ — 3. Rule 3
4. $\log(\log \sqrt{n}) = \log(\log n) - 1$ — 4. Rules 2, 3
5. $\log_4 n = \frac{\log n}{2}$ — 5. Rule 4
6. $\log 2^n = n$ — 6. Rule 3

# Examples 2/2

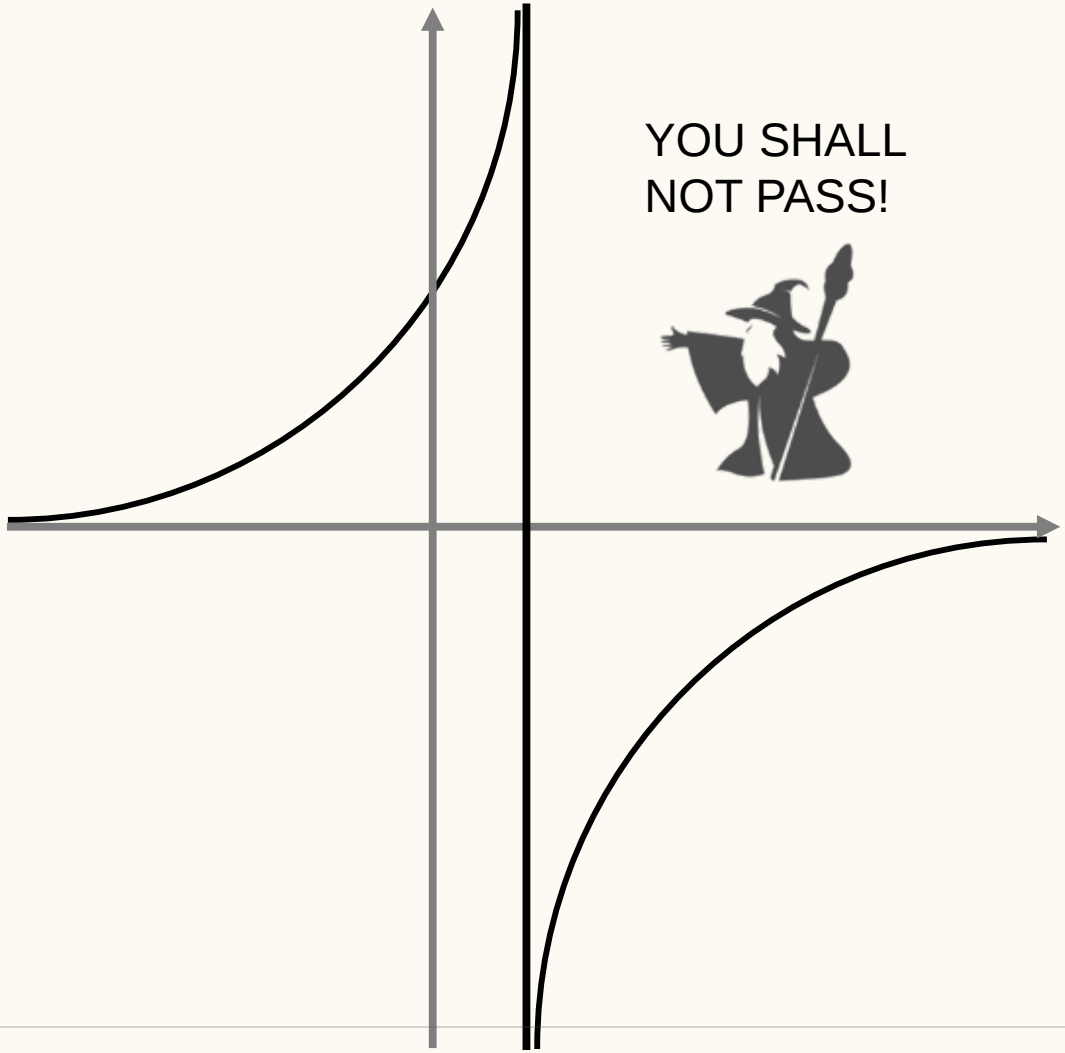
7. $2^{\log n} = n$ — 7. Rule 5
8. $2^{2 \log n} = n^2$ — 8. Rule 6
9. $4^n = 2^{2n}$ — 9. Rule 5
10. $n \cdot 2^{(3 \log n)} = n^3 * n^2 = n^5$ — 10. Rules 5, 6
11. $\frac{4^n}{2^n} = 2^n$ — 11. Rules 5, 6, 7

12. Rules 6, 8

# ASYMPTOTE

# ASYMPTOTE

An asymptote is a line that a curve approaches as it tends towards infinity or a specific value, but **never actually touches**.



# POLYNOMIAL

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0 x^0 = \sum_{k=0}^{n} a_k x^k$$

where $a_0, \ldots, a_n$ are constants called the coefficients of the polynomial, and $x$ is a variable.

**Weierstrass Approximation Theory**

> Any continuous function on a closed interval can be approximated uniformly by a polynomial to any desired degree of accuracy.
> In other words, we can find a polynomial that will "draw" the graph of our function as closely as we want on that interval.

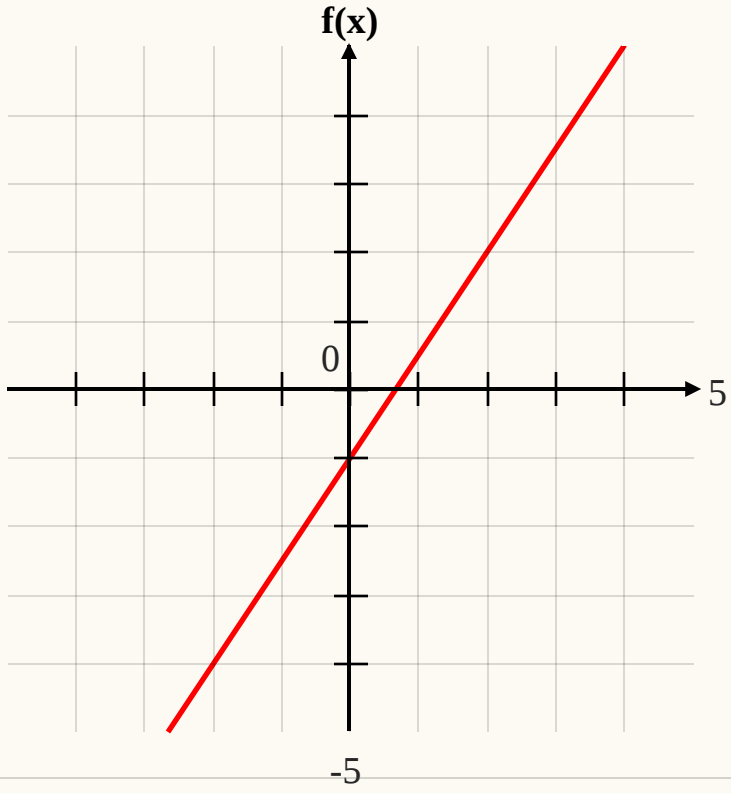
# A LINEAR FUNCTION

A linear function is a special case of a polynomial function:

$$f(x) = ax + b \equiv a_1 x^1 + a_0 x^0$$

The **slope** (gradient) of a linear function is directly influenced by the coefficient of the x-term.

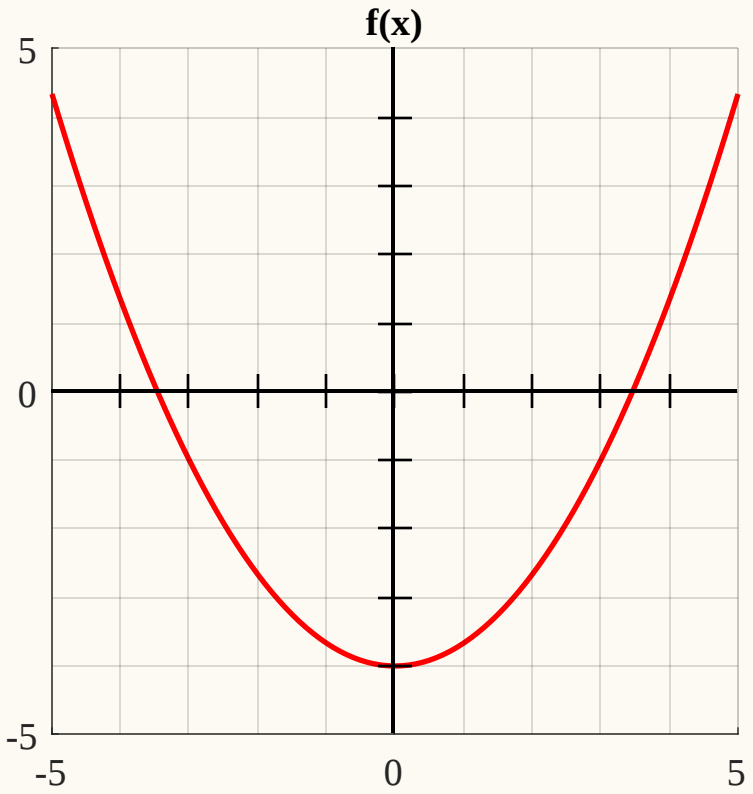
The greater the absolute value of slope, the **steeper** the graph of the function will be.

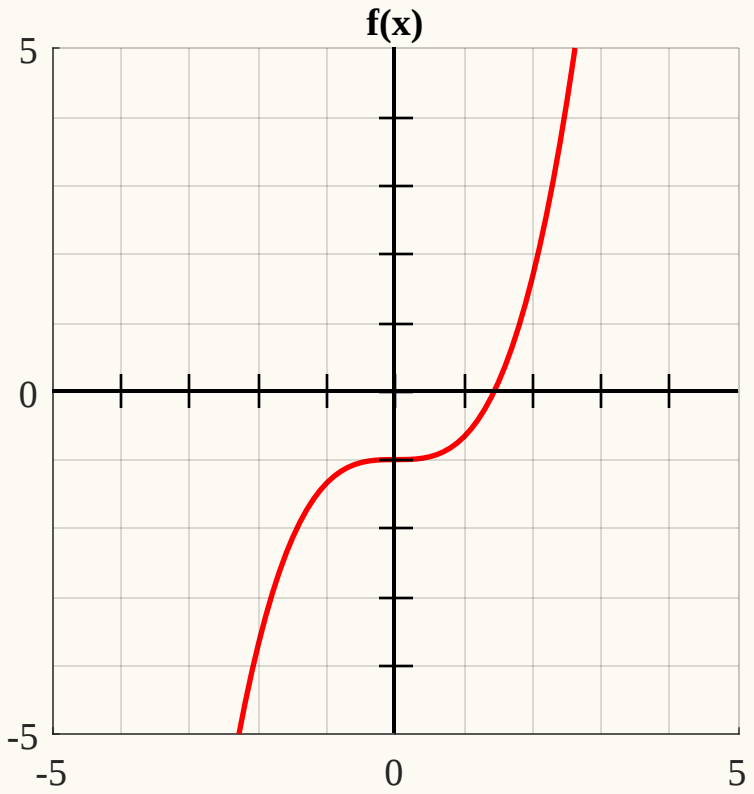


# A Quadratic Function & Cubic Polynomial

$$f(x) = ax^2 + bx + c \equiv a_2x^2 + a_1x^1 + a_0x^0$$

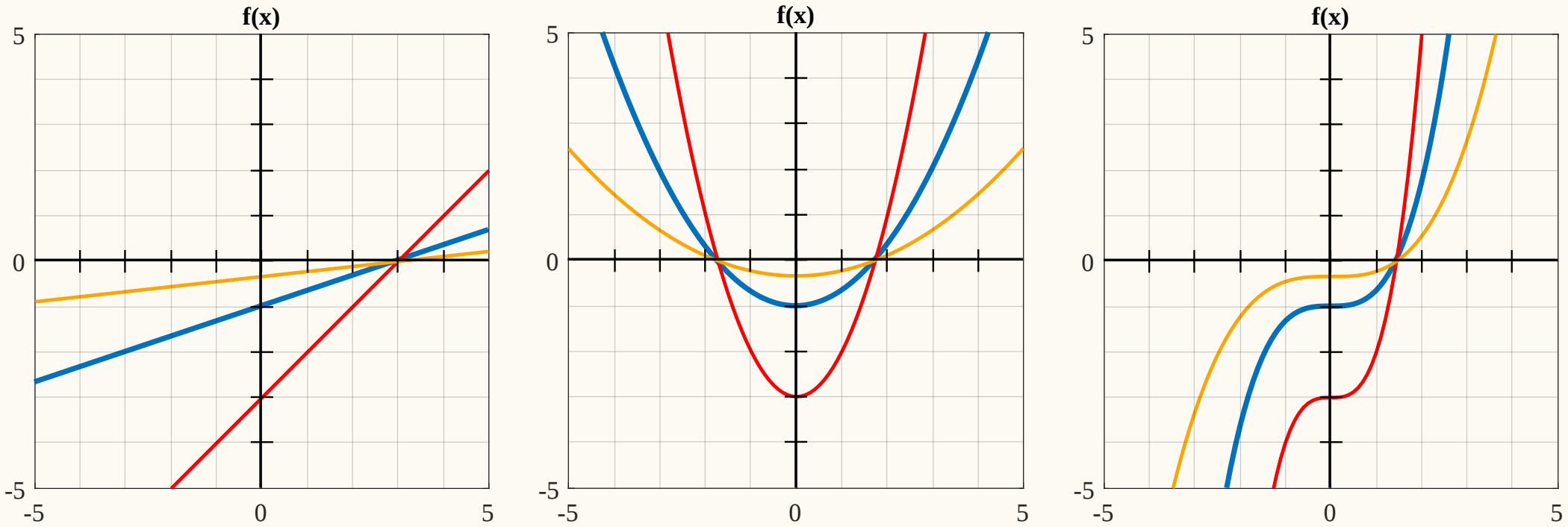
$$f(x) = ax^3 + bx^2 + cx + d \equiv a_3x^3 + a_2x^2 + a_1x^1 + a_0x^0$$




As the **coefficient** increases, the rate of growth of the values will accelerate.

# Multiplication of a Function by a Constant c




**If $C > 1$:** The graph will be **stretched vertically**. The larger C, the narrower and taller the graph becomes.

**If $0 < C < 1$:** The graph will be **compressed vertically**. The closer C is to zero, the wider and flatter the graph becomes.

# ASYMPTOTIC

## Notation

# ASYMPTOTIC NOTATION

- The **order of growth** of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and allows us to compare the relative performance of alternative algorithms.
- When we look at input sizes large enough to make only the order of growth relevant, we are studying the **asymptotic efficiency** of algorithms.
- We are concerned with how the running time of an algorithm increases with the size of the input **in the limit**, as the size of the input increases without bound. Usually, an algorithm that is asymptotically more efficient will be the best choice for all but very small inputs.

# ASYMPTOTIC NOTATION

- The notations we use to describe the asymptotic running time are defined in terms of functions whose domains are the set of natural numbers $N = 0, 1, 2, \ldots$.
- Such notations are convenient for describing **the worst-case running-time function** $T(n)$, which usually is defined only on integer input sizes.
- Asymptotic notation actually applies to functions. For example, if a sorting algorithm has the worst-case running time of $(ax^2 + bx + c)$, where a, b, and c are constants, then the running time is $\Theta(n^2)$.

# ASYMPTOTIC NOTATION — THE SET DEFINITION

- $T(n) = \Theta(n^2)$
- Let us define what this notation means.
- For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions:

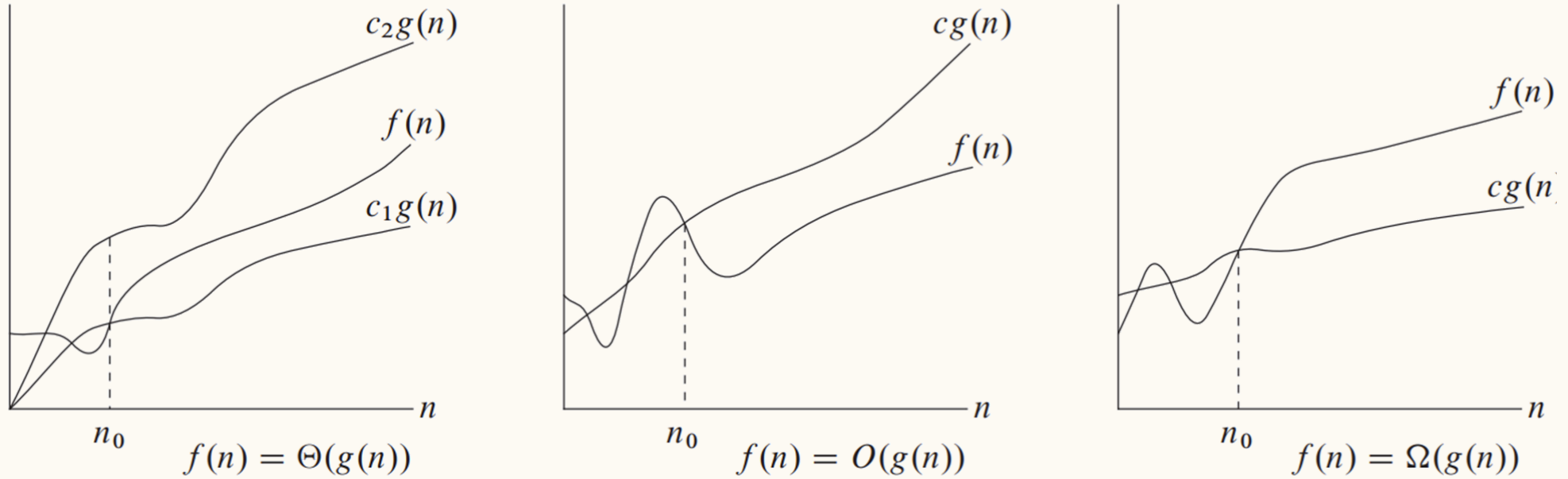
$$\Theta(g(n)) = \{\ f(n)\ :\ \text{there exist positive constants } c_1,\ c_2,\ n_0\ \text{ such that}$$
$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{for all } n \geq n_0\ \}$$

# ASYMPTOTIC

## Θ-Notation

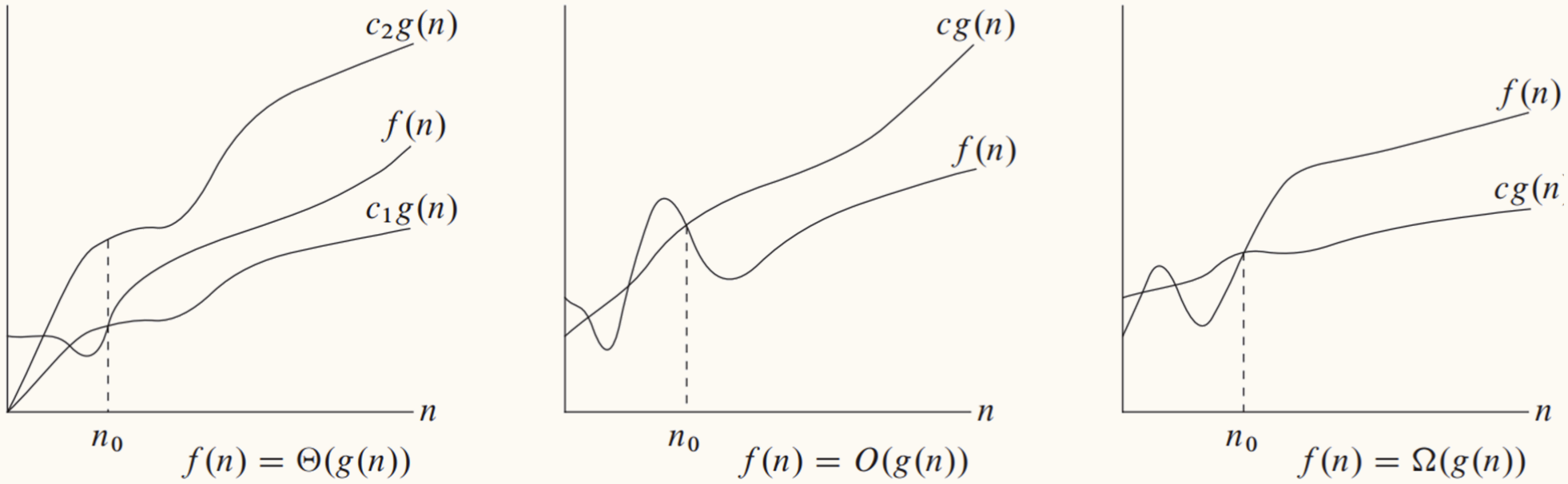
# Θ-NOTATION — VISUAL OVERVIEW

$$\Theta(g(n)) = \{\, f(n) \;:\; \exists\, c_1, c_2, n_0 > 0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n)\ \forall\, n \ge n_0 \,\}$$




Left: $f(n)$ is **sandwiched** between $c_1 g(n)$ and $c_2 g(n)$ — tight bound.
Middle: $f(n)$ stays **below** $cg(n)$ — upper bound only.
Right: $f(n)$ stays **above** $cg(n)$ — lower bound only.

# Θ-Notation — Intuition




A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants $c_1$ and $c_2$ such that it can be **"sandwiched"** between $c_1 g(n)$ and $c_2 g(n)$, for sufficiently large $n$. Because $\Theta(g(n))$ is a set, we could write "$f(n) \in \Theta(g(n))$" — instead, we usually write "$f(n) = \Theta(g(n))$" to express the same notion.

# Θ-Notation — Asymptotically Tight Bound

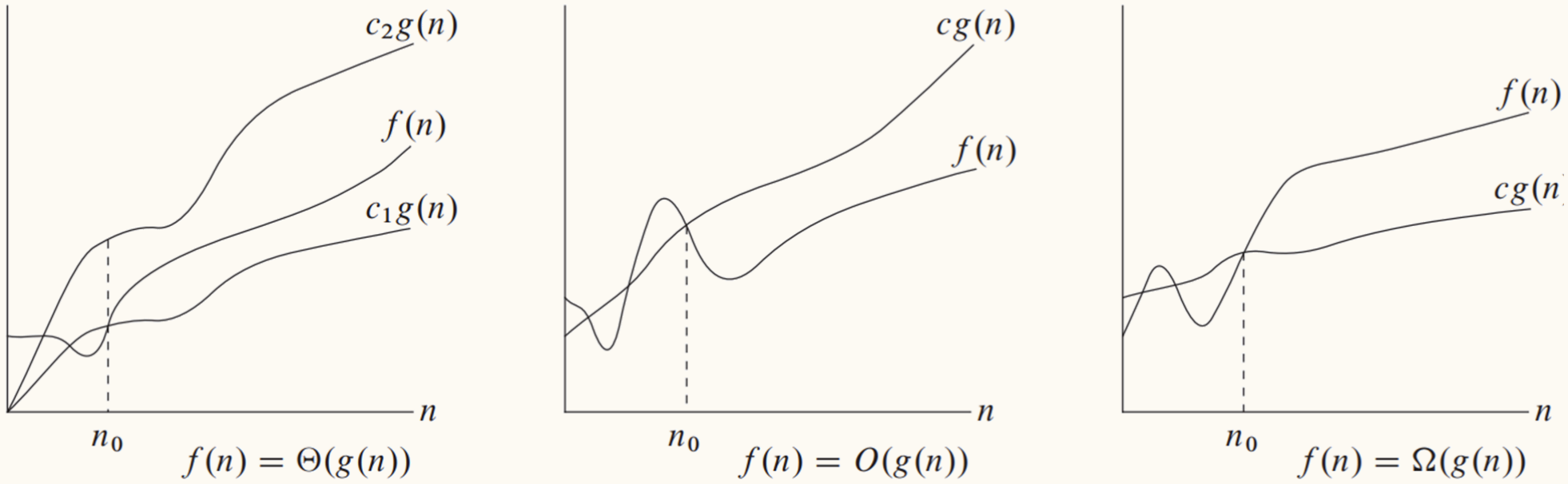
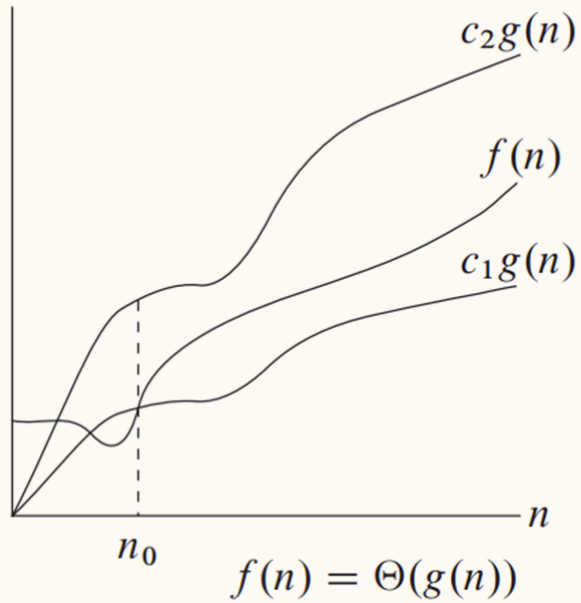
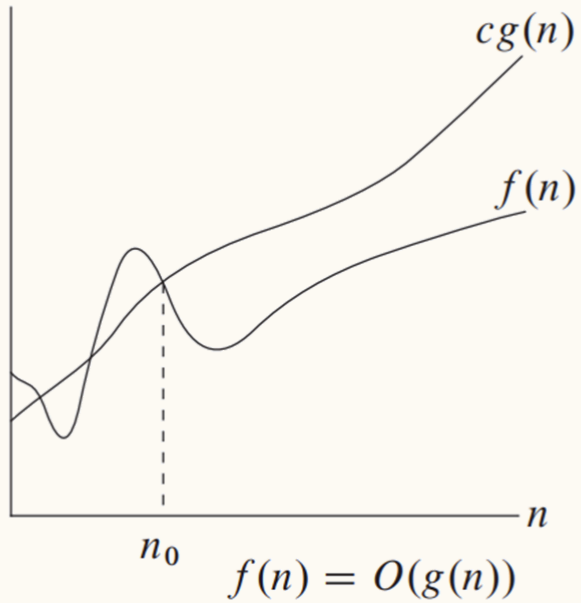
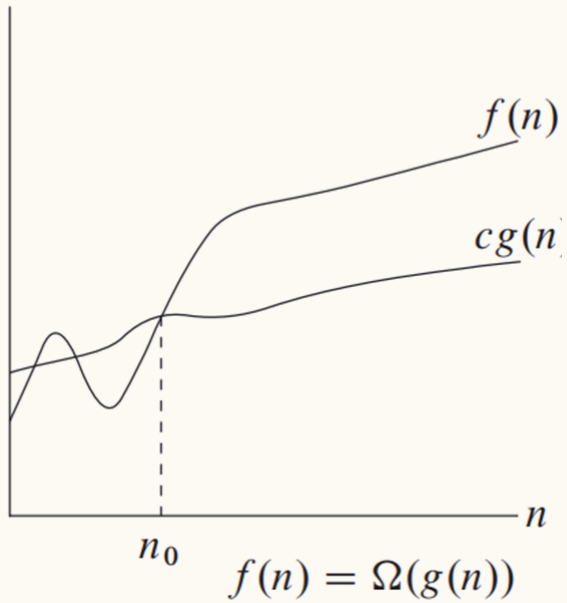



Figure (a) gives an intuitive picture of functions $f(n)$ and $g(n)$, where $f(n) = \Theta(g(n))$. For all values of n at and to the right of $n_0$, the value of $f(n)$ lies at or above $c_1g(n)$ and at or below $c_2g(n)$. In other words, for all $n \geq n_0$, the function $f(n)$ is equal to $g(n)$ **to within a constant factor**. We say that $g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

# Θ-NOTATION — ASYMPTOTICALLY NONNEGATIVE



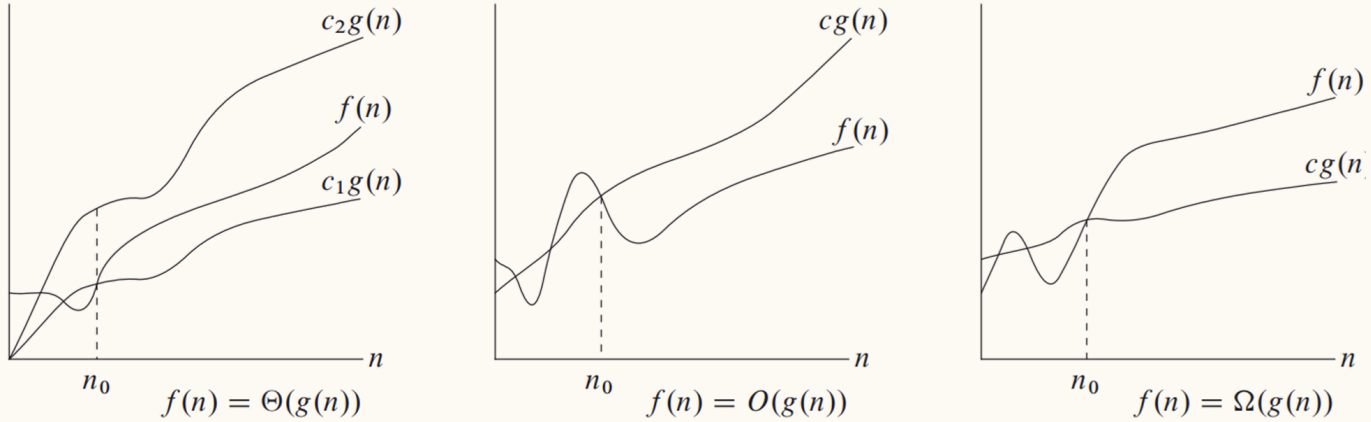




The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be ***asymptotically nonnegative***, that is, that $f(n)$ be nonnegative whenever $n$ is sufficiently large. An ***asymptotically positive*** function is one that is positive for all sufficiently large $n$.

# ASYMPTOTIC NOTATION

## O-Notation

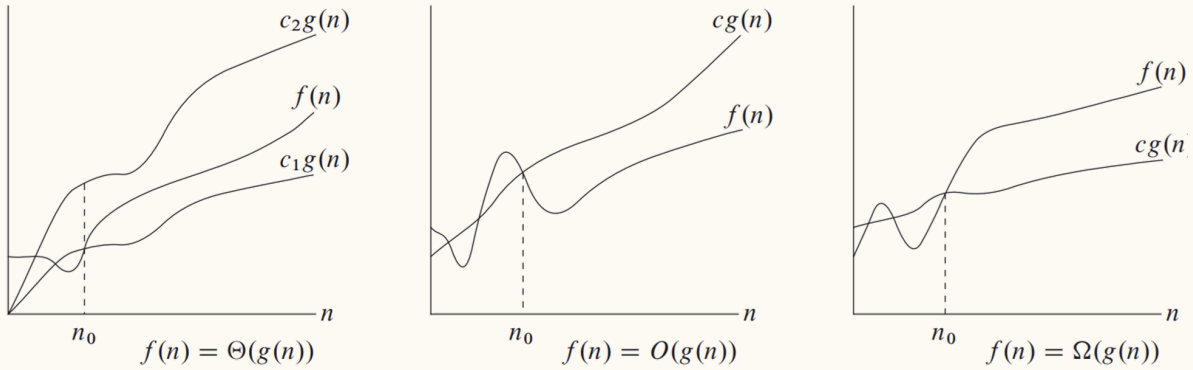
# O-Notation — Asymptotic Upper Bound




The Θ-notation asymptotically bounds a function from **above and below**. When we have only an ***asymptotic upper bound***, we use O-notation. For a given function $g(n)$, we denote by $O(g(n))$ (pronounced "big-oh of g of n") the set of functions:

$$O(g(n)) = \{\ f(n)\ :\ \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$$
$$0 \le f(n) \le cg(n) \quad \text{for all } n \ge n_0\ \}$$

# O-Notation - Upper Bound to Within a Constant

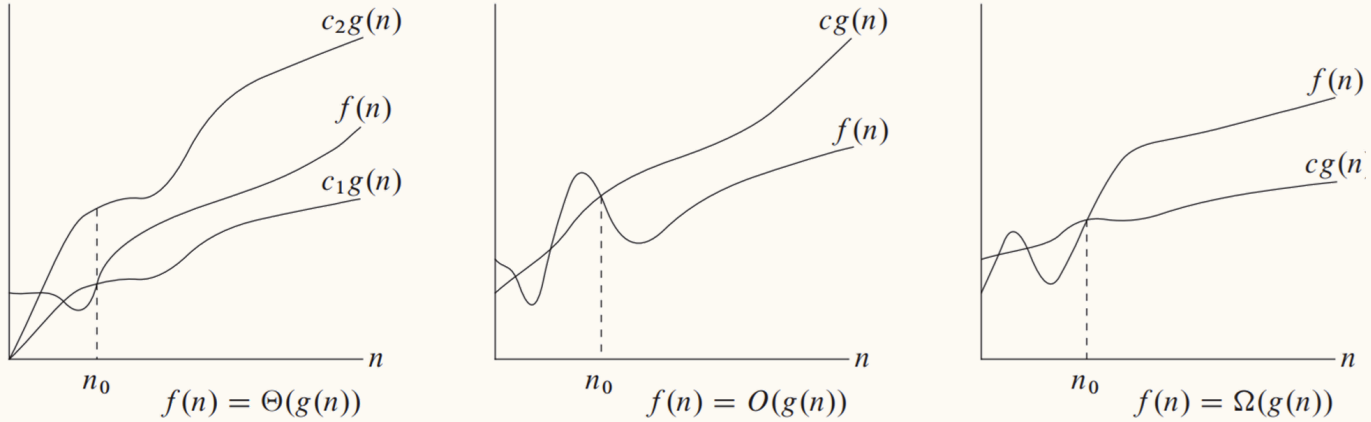



We use O-notation to give an **upper bound** on a function, to within a constant factor. For all values $n$ at and to the right of $n_0$, the value of the function $f(n)$ is on or below $cg(n)$. We write $f(n) = O(g(n))$ to indicate that a function $f(n)$ is a member of the set $O(g(n))$. Note that $f(n) = \Theta(g(n)) implies f(n) = O(g(n))$, since Θ-notation is a stronger notion than O-notation.

# ASYMPTOTIC NOTATION

## Ω-Notation

# Ω-Notation — Asymptotic Lower Bound




Just as O-notation provides an asymptotic *upper* bound on a function, **Ω-notation provides an *asymptotic lower bound***. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced "big-omega of g of n") the set of functions:

$$\Omega(g(n)) = \{\ f(n)\ :\ \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$$
$$0 \le cg(n) \le f(n) \quad \text{for all } n \ge n_0\ \}$$

# Ω-NOTATION — LOWER BOUND TO WITHIN A CONSTANT

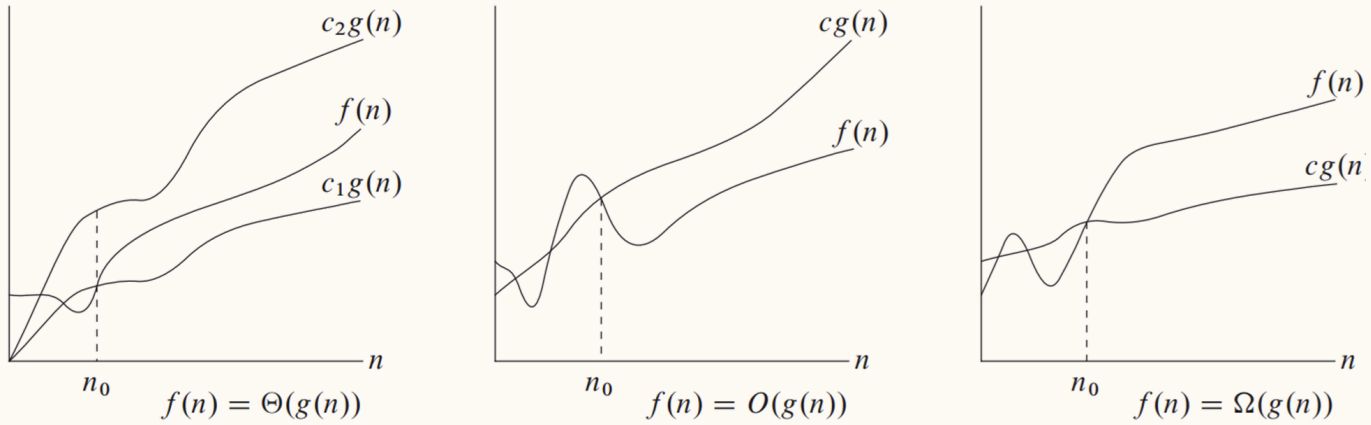



Figure (c) shows the intuition behind Ω-notation. For all values $n$ at or to the right of $n_0$ , the value of $f(n)$ is **on or above** c$g(n)$. From the definitions of the asymptotic notations we have seen thus far, it is easy to prove the following important theorem:

> **Theorem:** For any two functions $f(n)$ and $g(n)$: > $f(n) = \Theta(g(n))$ **if and only if** $f(n) = O(g(n)) and f(n) = \Omega(g(n))$.

# EXAMPLE

# EFFICIENCY — AN EXAMPLE

Let us pit a faster computer **A** running insertion sort $O(n^2)$ against a slower computer **B** running merge sort $O(nlgn)$.

They each must sort an array of **10 million numbers**. If the numbers are eight-byte integers, the input occupies about 80 megabytes — fits in memory of even an inexpensive laptop.

- Computer **A** executes **10 billion instructions per second**
- Computer **B** executes only **10 million instructions per second**
- Computer **A** is **1000 times faster** than computer **B** in raw computing power

# EFFICIENCY — AN EXAMPLE

The world's craftiest programmer codes insertion sort in machine language for computer **A** — resulting code requires $n^2$ instructions. An average programmer implements merge sort using a high-level language with an inefficient compiler — resulting code takes 50n lg n instructions.

To sort **10 million numbers**, computers take:

$$\text{A:} \quad \frac{2 \cdot (10^7)^2}{10^{10}} \frac{\text{instructions}}{\text{instructions/second}} = 20{,}000 \text{ seconds} \quad (\text{more than } 5.5 \text{ hours})$$

$$\text{B:} \quad \frac{50 \cdot \log_2(10^7)}{10^7} \frac{\text{instructions}}{\text{instructions/second}} = 1163 \text{ seconds} \quad (\text{less than } 20 \text{ minutes})$$

# EFFICIENCY — CONCLUSION

By using an algorithm whose running time grows more slowly, even with a **poor compiler**, computer **B** runs more than **17 times faster** than computer **A**.

The advantage of merge sort is even more pronounced when we sort **100 million numbers**:

| | Insertion Sort $O(n^2)$ | Merge Sort $O(n \lg n)$ |
|---|---|---|
| 10 million | 5.5 hours | 20 minutes |
| 100 million | **more than 23 days** | **under 4 hours** |

> In general, as the problem size increases, so does the relative advantage of merge sort. **The algorithm matters more than the hardware.**

# THANK

You